



Lista dei Tool MCP

I 18 tool esposti dal server MCP (`codeql_mcp_server.py`) e usati dal modello Ollama. Pagina di riferimento, complementare a [Setup Guide](#) e [Query Templating & CWE KB](#).

Flusso tipico

```
list_python_files → create_codeql_database
  → analyze_database + check_* (taint / crypto / config)
  → cwe_knowledge
  → extract_api_surface → [LLM classifica API] → run_hypothes
is_query ← spec inference + isBarrier
  oppure run_taint_query / run_api_misuse_query
← template name-based
  oppure run_official_query
← official / config
  → run_inspection_query
← debug source/sink/barrier
  → read_file_snippet → REPORT
```

1. Orchestrazione / setup

Tool	Parametri	Cosa fa
list_python_files	<code>repo_path</code> , <code>max_files=500</code>	Elenca i file <code>.py</code> della repo (salta venv/site-packages/.git). Permette al modello di ragionare "file per file".
create_codeql_database	<code>repo_path</code> , <code>language=python</code>	Costruisce il database CodeQL dalla repo. Ritorna il <code>db_path</code> (in <code>_work/</code>). Passo obbligatorio prima di ogni analisi.

2. Analisi standard

Tool	Parametri	Cosa fa
analyze_database	<code>db_path</code> , <code>suite=""</code>	Esegue la suite di sicurezza standard (<code>python-security-extended.qls</code>) e ritorna i finding parsati (regola, severità, CWE, file, riga, flow path). Copertura ampia.
run_custom_query	<code>db_path</code> , <code>query_path</code>	Esegue un file <code>.ql</code> arbitrario indicato per path. Per query ad-hoc fuori dal registry.

3. Check curati — Taint (query fisse)

Query pre-scritte per le classi injection più comuni. Il modello le usa come check "a un clic".

Tool	CWE	Cosa rileva
check_sql_injection	CWE-89	SQL injection (incl. wrapper DB custom)
check_os_command_injection	CWE-78	OS command injection
check_path_traversal	CWE-22	Path traversal
check_deserialization	CWE-502	Deserializzazione insicura
check_xss	CWE-79	Cross-site scripting

(Tutti prendono un solo parametro: `db_path` .)

4. Check curati — Crypto (query fisse)

Rilevamento di API crittografiche deboli (point detection). La suite standard spesso le manca, quindi vanno eseguite sempre.

Tool	CWE	Cosa rileva
check_weak_hash	CWE-328	Hash deboli (MD5, SHA-1)
check_broken_crypto	CWE-327	Cifrari rotti/rischiosi (DES, RC4, ECB)
check_weak_random	CWE-330	Randomness insufficiente (modulo <code>random</code> per scopi di sicurezza)

Tool	CWE	Cosa rileva
check_weak_password_hash	CWE-916	Hash password senza KDF/salt

(Tutti prendono un solo parametro: `db_path`.)

5. Query parametriche — dall'agente al CodeQL

Due modalità: **name-based** (template fisso, iter 0) e **hypothesis-driven** (spec LLM → Customizations dinamiche, iter 2). In entrambi i casi l'agente NON scrive CodeQL: passa parametri strutturati, il server MCP genera il `.ql`.

Tool	Parametri	Cosa fa
run_taint_query	<code>db_path</code> , <code>sink_names</code> (obbl.), <code>cwe</code> , <code>source_names?</code> , <code>sanitizer_names?</code>	Verifica un'ipotesi di taint (source → sink). Trova flussi verso wrapper custom che le query standard mancano. Ritorna il path completo source → sink come evidenza.
run_api_misuse_query	<code>db_path</code> , <code>weak_call_names?</code> , <code>bad_constants?</code> , <code>cwe</code>	Verifica un'ipotesi di API insicura (crypto). Point detection: flagga chiamate a API deboli per nome e/o con argomento-costante debole (es. <code>hashlib.new("md5")</code>).

6. Knowledge base CWE

Tool	Parametri	Cosa fa
list_cwes	—	Elenca le classi CWE note (16) con <code>detection</code> (taint/api_misuse) e il tool giusto da usare.
cwe_knowledge	<code>cwe</code>	"Pagina wiki" del CWE in 3 sezioni: <code>official</code> (descrizione + mitigazioni MITRE), <code>our_knowledge</code> (intuizione + remediation), <code>actions</code> (playbook: tool, nomi candidati, come interpretare). Da consultare PRIMA di formulare un'ipotesi.

7. Ispezione

Tool	Parametri	Cosa fa
read_file_snippet	<code>file_path</code> , <code>start_line=1</code> , <code>end_line=40</code>	Legge un intervallo di righe sorgente per ispezionare il contesto di un finding (con numeri di riga).

Note

- I `check_*` (taint e crypto) sono generati automaticamente: i taint dal registry `CUSTOM_QUERIES` , i crypto dal wiki (`CRYPTO_CHECK_SLUGS`).
- `run_taint_query` / `run_api_misuse_query` scrivono il `.ql` generato in `generated_queries/` (nome = hash del contenuto).
- Ogni finding ritorna anche il `cwe` estratto deterministicamente dai tag SARIF della query.

6. Check — Config / Strutturale (via `run_official_query`)

Query **point detection** senza taint tracking: rilevano misconfigurazioni, flag mancanti, pattern insicuri. Nessuna spec source/sink necessaria — si eseguono direttamente con `run_official_query(db_path, cwe_id, query_name)` .

CWE	Query	Cosa rileva
CWE-215	<code>FlaskDebug</code>	Flask in modalità debug in produzione
CWE-295	<code>RequestWithoutValidation</code> , <code>MissingHostKeyValidation</code>	HTTPS senza verifica certificato / host key
CWE-312	<code>CleartextLogging</code> , <code>CleartextStorage</code>	Dati sensibili in chiaro in log / storage
CWE-352	<code>CSRFProtectionDisabled</code>	CSRF disabilitato (Django/Flask)
CWE-377	<code>InsecureTemporaryFile</code>	File temporanei con <code>mktemp</code>
CWE-614	<code>InsecureCookie</code>	Cookie senza flag <code>Secure</code>
CWE-730	<code>ReDoS</code> , <code>PolynomialReDoS</code>	Regex vulnerabili a Denial of Service

CWE	Query	Cosa rileva
CWE-732	<code>WeakFilePermissions</code>	Permessi file troppo aperti
CWE-776	<code>XmlBomb</code>	Entità XML espansive (Billion Laughs)
CWE-798	<code>HardcodedCredentials</code>	Credenziali hardcoded nel codice
CWE-1004	<code>NonHttpOnlyCookie</code>	Cookie senza flag <code>HttpOnly</code>
CWE-1275	<code>SameSiteNoneCookie</code>	<code>SameSite=None</code> senza <code>Secure</code>
CWE-020	<code>CookieInjection</code> , <code>IncompleteHostnameRegExp</code>	Cookie injection, hostname regex incompleta
CWE-209	<code>StackTraceExposure</code>	Stack trace esposto all'utente

7.5 Nuovi tool — Spec Inference & Hypothesis-Driven iter 2)

L'architettura **hypothesis-driven**: l'LLM osserva il codice, genera un'ipotesi `{sources, sinks, sanitizers}`, il server MCP la traduce in classi `*Customizations.ql` con `isBarrier` dinamico e lancia la query ufficiale. CodeQL dà la risposta binaria deterministica.

Tool	Parametri	Cosa fa
extract_api_surface	<code>repo_path</code>	Esegue <code>CWE-020-ExternalAPIs/UntrustedDataToExternalAPI.ql</code> : trova tutte le API esterne che ricevono dato untrusted. Output: lista <code>(api_name, location)</code> → input per la classificazione LLM (stage 1 spec inference).
run_hypothesis_query	<code>db_path</code> , <code>cwe</code> , <code>sources[]</code> , <code>sinks[]</code> , <code>sanitizers[]</code>	Genera dinamicamente <code>*InferredCustomizations.ql</code> con classi che estendono <code>Source / Sink / Sanitizer</code> delle <code>*Customizations.ql</code> ufficiali. Esegue la query ufficiale con <code>--search-path</code> che include il file generato. Il server scrive il QL — non l'agente. Fallback su template name-based se le liste sono vuote.

Tool	Parametri	Cosa fa
run_official_query	<code>db_path</code> , <code>cwe_id</code> , <code>query_name</code>	Esegue direttamente una query ufficiale dalla libreria (<code>Security/CWE-xxx/QueryName.ql</code>) senza spec custom. Copre tutte le famiglie non-taint (Config/Strutturale) e le query taint ufficiali senza estensioni.
run_inspection_query	<code>db_path</code> , <code>cwe_id</code> , <code>type</code>	Esegue le query di ispezione (<code>Sql_source.ql</code> , <code>Sql_sink.ql</code> , <code>sanitizer.ql</code> , ecc.) per vedere cosa CodeQL riconosce attualmente per un CWE. Debug: confronta prima/dopo l'iniezione delle Customizations generate.

8. Famiglie di query — mappa CWE → meccanismo → tool

Famiglia	Meccanismo CodeQL	CWE coperti	Tool Sibyl
Taint / injection	<code>path-problem</code> , Source → Sink+ <code>isBarrier</code> , estensibile via <code>*Customizations.ql</code>	022, 074, 078, 079, 089, 090, 094, 113, 117, 502, 601, 611, 643, 730(regex), 918, 943	<code>run_taint_query</code> (name-based), <code>run_hypothesis_query</code> (spec inference), <code>check_*</code> curati
Crypto / API misuse	point detection su call site + argomenti costanti	326, 327, 328, 330, 916	<code>run_api_misuse_query</code> , <code>check_*</code> crypto
Config / Strutturale	point detection, flag/config check, nessun taint	020, 209, 215, 295, 312, 352, 377, 614, 730(ReDoS), 732, 776, 798, 1004, 1275	<code>run_official_query</code>
Extraction utility	scan API surface (CWE-020-ExternalAPIs)	—	<code>extract_api_surface</code> → input per spec inference

Pattern isBarrier: la famiglia Taint è l'unica dove `isBarrier` / `Sanitizer` determina i falsi positivi. Le famiglie Crypto e Config/Strutturale non hanno taint path — non serve spec inference, basta `run_official_query` .